

MANAGER ROUND INTERVIEW CHEAT SHEET

DevOps Engineer | AWS · Kubernetes · CI/CD · Terraform · Linux · Ansible

CHILUKURI RAJESH — 4+ Years Experience

83%

Deployment Time
Reduced

5 min

Deploy Time (was 30
min)

10 min

Infra Provision (was 3+
hrs)

0

Manual Deploy Errors

4+

Years Experience

1. TELL ME ABOUT YOURSELF (Practice 3x Tonight!)

"I'm Rajesh, a DevOps Engineer with 4+ years of experience at Accenture and L&T; Technology Services. I specialise in CI/CD pipelines using Jenkins and ArgoCD, Kubernetes on AWS EKS, and infrastructure automation with Terraform. My biggest win was reducing deployment time from 30 minutes to under 5 minutes — an 83% improvement — while eliminating all manual deployment errors. I work closely with dev teams to ship reliable software faster."

■ *Keep it under 60 seconds. Start with experience → tools → achievement → what you bring.*

2. BEHAVIOURAL / SITUATIONAL QUESTIONS

Q: What is your biggest achievement?

"At Accenture I built an end-to-end Jenkins pipeline with Docker, EKS, and ArgoCD GitOps. Deployment time dropped from 30 minutes to under 5 minutes (83% improvement) and we achieved zero manual deployment errors. The team could release multiple times a day confidently."

■ *Always say the number — 83%, 5 min, zero errors. Numbers impress managers.*

Q: Tell me about a challenging situation you handled.

"At L&T; I set up the full observability stack from scratch on Kubernetes. No dashboards existed and the team had zero visibility. I deployed Prometheus + Grafana, built dashboards for CPU/memory/pod health, and set up Alertmanager with Slack notifications. After that, the team detected incidents proactively instead of reacting after users complained."

■ *Shows ownership + problem solving — what managers love.*

Q: Have you ever made a mistake? What happened?

"Early on I applied a Terraform change in production without fully verifying the plan. It caused a brief disruption. I rolled back quickly, documented the incident, and from then on enforced: always run terraform plan, peer review, and apply during low-traffic windows. That incident made me a more careful engineer."

■ *Managers want self-awareness, not perfection.*

Q: How do you work with development teams?

"I treat DevOps as a collaboration. At Accenture and L&T; I worked with devs to resolve deployment blockers, helped them understand Kubernetes manifests, and built pipelines that gave fast feedback on their code. When issues arose I sat with the dev team, understood their pain points, and automated those steps."

■ *Shows team-player attitude — critical for manager round.*

Q: Why should we hire you?

"Because I solve business problems — not just set up tools. I reduced deployment time 83%, cut infra provisioning from 3 hours to 10 minutes, and achieved zero-downtime deployments. I'm hands-on with AWS, Kubernetes, Terraform, Jenkins, and Prometheus, and I take ownership of results."

■ *Confident, backed by numbers, no fluff.*

Q: Where do you see yourself in 2-3 years?

"Growing into a Senior DevOps or Platform Engineer role — architecting larger-scale solutions, mentoring junior engineers, and driving DevOps culture across teams. I also want to deepen expertise in cloud-native technologies."

3. CI/CD PIPELINE (Walk Through Confidently)

Q: Walk me through your CI/CD pipeline.

"Developer pushes code to GitHub → GitHub Webhook triggers Jenkins → Jenkins runs unit tests, builds Docker image, pushes to Amazon ECR → ArgoCD detects the change in GitHub and automatically deploys to Kubernetes EKS → If deployment fails, ArgoCD rolls back automatically using liveness probes. Whole process: under 5 minutes."

■ *Draw it on whiteboard if they offer. It shows confidence.*

Q: What is GitOps / ArgoCD?

"GitOps means Git is the single source of truth for your infrastructure and deployments. ArgoCD watches the GitHub repo and automatically syncs the Kubernetes cluster to match. If someone manually changes something in the cluster, ArgoCD detects the drift and corrects it. This gives us auditable, self-healing deployments."

4. TERRAFORM

Q: Tell me about your Terraform experience.

"I've used Terraform to provision AWS infrastructure — VPCs, subnets, EC2, IAM roles, security groups. I wrote modular, reusable Terraform code using S3 remote backend with DynamoDB for state locking so teams could work safely. I also integrated Terraform into Jenkins pipelines. This cut provisioning from 3+ hours to under 10 minutes."

Q: What is Terraform remote state and why does it matter?

"Remote state stores the Terraform state file in S3 instead of locally, so the whole team shares the same view of infrastructure. DynamoDB state locking prevents two people from running terraform apply at the same time and corrupting the state. It's essential for team-based infra management."

Q: Terraform vs Ansible — what's the difference?

"Terraform creates infrastructure (EC2, VPCs, S3). Ansible configures what's inside that infrastructure (installs software, deploys apps, manages users). I used both together — Terraform to provision AWS resources, then Ansible to configure them."

■ *This is a very common question — nail this answer!*

5. LINUX

Q: How comfortable are you with Linux?

"Very comfortable. All my work at Accenture and L&T; was on Ubuntu and Amazon Linux — managing servers, writing Bash scripts, checking logs, troubleshooting services, and working with Docker and Kubernetes."

Command	Use / Example
<code>systemctl status nginx</code>	Check if a service is running
<code>journalctl -u nginx</code>	View service logs to find root cause
<code>lsof -i :8080</code>	Find which process is using port 8080
<code>kill -9 <PID></code>	Force-stop a process
<code>df -h</code>	Check disk space (human readable)
<code>free -m</code>	Check memory usage in MB
<code>top / htop</code>	Real-time CPU usage
<code>grep -i 'error' app.log</code>	Search for errors in log file
<code>tail -f app.log</code>	Watch logs in real time
<code>chmod 600 key.pem</code>	Restrict file permissions (owner only)
<code>chown appuser:appuser /app</code>	Change file owner
<code>crontab -e</code>	Edit scheduled cron jobs
<code>0 2 * * * /scripts/cleanup.sh</code>	Cron: run cleanup every day at 2 AM

6. ANSIBLE

Q: What is a Playbook?

"A YAML file that defines what tasks Ansible should run on which servers. For example, I wrote a playbook to install Docker on all application servers, configure it, and start the service — across 10 servers simultaneously in one run. Without Ansible I'd SSH into each server manually."

Q: What is Ansible Inventory?

"A list of servers Ansible manages. It can be static (IPs in a file) or dynamic. I used dynamic inventory with the AWS EC2 plugin — when Auto Scaling launched new servers, Ansible automatically knew about them without any manual updates."

Q: Is Ansible agentless? Why does that matter?

"Yes — Ansible connects over SSH, no software needed on target servers. Nothing to maintain or update on managed servers. Compare this to Chef or Puppet which require an agent running on every server."

Q: What is idempotency in Ansible?

"Running the same Ansible task 10 times gives the same result — no side effects. If Nginx is already installed, Ansible won't reinstall it. This makes automation safe to re-run anytime."

Concept	Simple Explanation
Task	One single action: 'install nginx'
Play	Group of tasks for specific servers
Playbook	File containing one or more plays
Inventory	List of servers Ansible manages
Role	Reusable, organized collection of tasks
Idempotent	Safe to run multiple times — same result
Agentless	Uses SSH — no agent on target servers

7. QUESTIONS TO ASK THE MANAGER (Always Ask 2!)

- "What does the current DevOps setup look like and what's the biggest pain point for the team?"
- "What would success look like for me in the first 90 days?"
- "How does the team handle on-call and incident response today?"
- "What is the deployment frequency and how mature is the CI/CD pipeline?"

8. CONFIDENCE TIPS FOR TODAY

DO ■	DON'T ■
Use STAR method (Situation → Task → Action → Result)	Blame teammates or managers
Quantify: 83%, 5 min, zero errors, 10 min	Give vague or theoretical answers
Show ownership — say 'I' not 'we'	Say 'I don't know' without offering to learn
Speak slowly — nervousness makes us rush	Panic if you don't know something
Smile — managers hire people they like	Leave without asking any questions
If stuck: 'I haven't done that yet but I learn quickly'	Over-explain or go off-topic

■ You have REAL achievements. Real numbers. Real experience.

Go in there, tell YOUR story confidently — YOU'VE GOT THIS, RAJESH! ■